

Relatório sobre a atividade da semana 03: Implementação de algoritmos de ordenação em C

Discente: Luiz Miguel Nascimento Ávila

Docente: Vinicius Sarmento

1. Objetivo da atividade:

Esta atividade teve como objetivo implementar e analisar o desempenho de diferentes algoritmos de ordenação em C. Foram implementados os algoritmos: BubbleSort, SelectionSort, InsertionSort, QuickSort e MergeSort. E os testes foram realizados com vetores de tamanho 100, 1000, 10000, utilizando números aleatórios.

2. Como os algoritmos foram implementados:

BubbleSort compara elementos adjacentes e realiza trocas quando necessário. Sua complexidade no pior caso é $O(n^2)$.

SelectionSort percorre todo o vetor buscando o menor elemento e o posiciona na posição correta. Sua complexidade é $O(n^2)$.

InsertionSort constrói o vetor ordenado inserindo cada elemento na posição correta dentro da parte já ordenada. No melhor caso possui complexidade $O(n)$.

QuickSort foi implementado utilizando recursão e divisão. Um elemento é escolhido como pivô e o vetor é posicionado. Possui complexidade $O(n \log n)$

MergeSort o vetor é dividido recursivamente e depois as partes são intercaladas de forma ordenada. Possui complexidade $O(n \log n)$.

3. Resultados dos testes:

Algoritmo	100 elementos	1000 elementos	10000 elementos
Bubble Sort	0.0000 s	0.004 s	0.34s
Selection	0.0000 s	0.002 s	0.18s
Insertion	0.0000 s	0.001 s	0.10s
Quick	0.0000 s	0.0000 s	0.004 s
Merge	0.0000 s	0.001 s	0.002s

***Em 100 elementos os tempos foram praticamente 0 pois são vetores pequenos devido à alta velocidade do processador e à baixa carga computacional.**

4. Comparação de Desempenho:

Se percebe que os algoritmos bubble, selection, e insertion tiveram um pior resultado com vetores maiores. Já o quick e merge foram significativamente mais rápidos com 10000 elementos, pois possuem complexidade $O(n \log n)$ e os demais $O(n^2)$. Assim, para volumes maiores de dados, os algoritmos baseados em divisão e conquista são mais eficientes.